

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

За седьмой семестр

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рубцов Д.А.

Проверила:

Андренко К.В.

Брест 2026


```

testset = torchvision.datasets.CIFAR10(root=data_path, train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=64,
                                       shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
           'truck')

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 32 * 8 * 8)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

net = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adadelta(net.parameters())

epochs = 10
loss_history = []

for epoch in range(epochs):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data[0].to(device), data[1].to(device)

        optimizer.zero_grad()

        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(trainloader)
    loss_history.append(epoch_loss)
    print(f'Epoch [{epoch + 1}/{epochs}], Loss: {epoch_loss:.4f}')

```

```

plt.plot(range(1, epochs + 1), loss_history, marker='o', color='b')
plt.title('Loss function')
plt.xlabel('Epoch')
plt.ylabel('Loss (Cross Entropy)')
plt.grid(True)
plt.show()
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data[0].to(device), data[1].to(device)
        outputs = net(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'Accuracy: {accuracy:.2f}%')
def imshow(img):
    img = img / 2 + 0.5
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

dataiter = iter(testloader)
images, labels = next(dataiter)

idx = np.random.choice(64, 4, replace=False)
sample_images = images[idx]
sample_labels = labels[idx]

imshow(torchvision.utils.make_grid(sample_images))
print('Real: ', ' '.join(f'{classes[sample_labels[j]]}' for j in range(4)))

sample_images = sample_images.to(device)
outputs = net(sample_images)
_, predicted = torch.max(outputs, 1)

print('Prediction: ', ' '.join(f'{classes[predicted[j]]}' for j in range(4)))

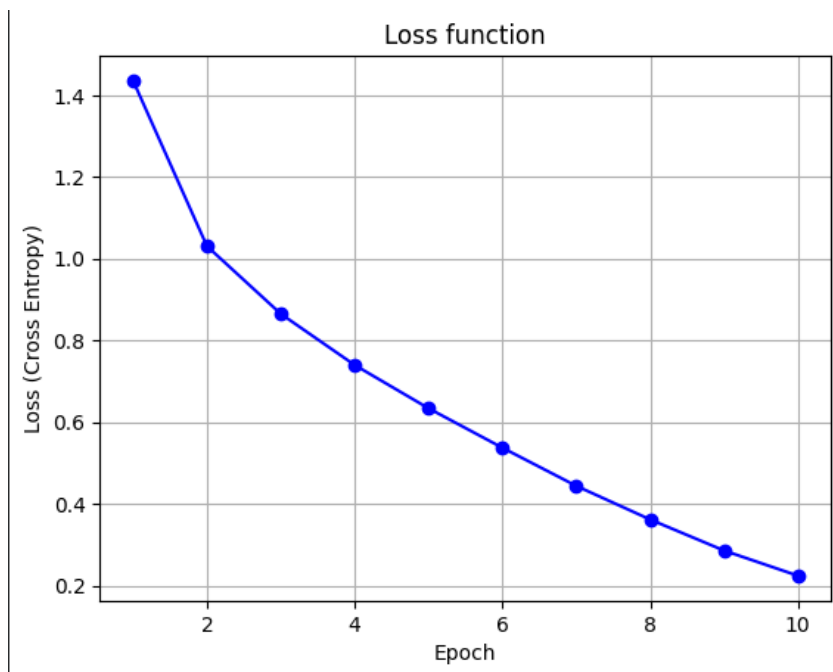
```

Результат:

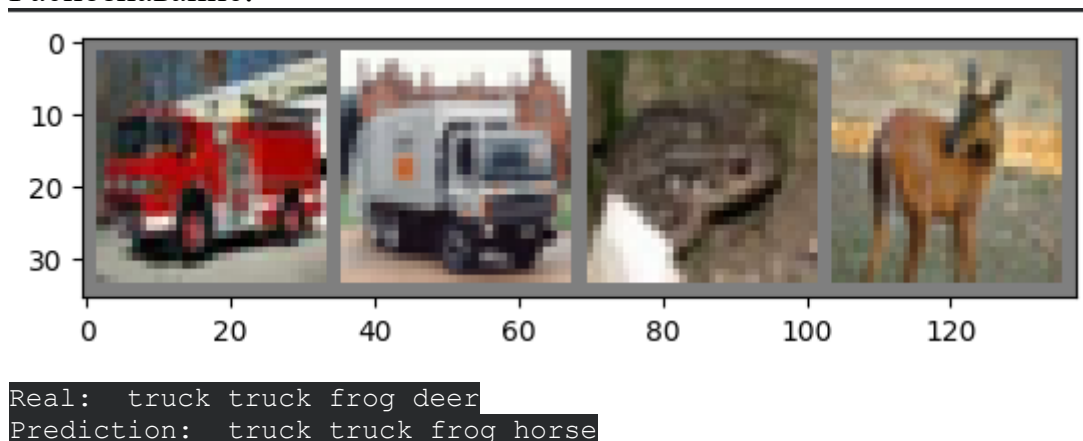
```

Epoch [1/10], Loss: 1.4357
Epoch [2/10], Loss: 1.0307
Epoch [3/10], Loss: 0.8650
Epoch [4/10], Loss: 0.7397
Epoch [5/10], Loss: 0.6339
Epoch [6/10], Loss: 0.5374
Epoch [7/10], Loss: 0.4440
Epoch [8/10], Loss: 0.3619
Epoch [9/10], Loss: 0.2859
Epoch [10/10], Loss: 0.2243

```



Распознавание:



Результат модели показал точность распознавания в пределах 69% такой результат обусловлен комплексными изображениями с их большим разрешением и наличием цвета что является сложной задачей для простой модели с двумя сверточными слоями. Если сравнивать с результатами SOTA моделей которые достигают точности в плоть до 99.5% такие результаты достигаются за счет пред обучения на огромных дата сетах по типу JFT-300M что занимает много времени по этому сеть обученная за 10 эпох распознающая больше трети изображений нормальный результат.

Вывод: научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.